JavaScript is disabled on your browser.

compact1, compact2, compact3

java.nio.file

Class Files

java.lang.Object
    java.nio.file.Files

```
public final class Files
extends Object
```

This class consists exclusively of static methods that operate on files, directories, or other types of files.

In most cases, the methods defined here will delegate to the associated file system provider to perform the file operations.

Since:

1.7

Method Summary

Concrete Methods

Modifier and Type	Method
static long	copy(InputStream in, Path target, CopyOption... options)
static long	copy(Path source, OutputStream out)
static Path	copy(Path source, Path target, CopyOption... options)

static **Path**	**createDirectories**(**Path** dir, **FileAttribute** >... attrs)
static **Path**	**createDirectory**(**Path** dir, **FileAttribute**<? >... attrs)
static **Path**	**createFile**(**Path** path, **FileAttribute**<? >... attrs)
static **Path**	**createLink**(**Path** link, **Path** existing)
static **Path**	**createSymbolicLink**(**Path** link, **Path** target **FileAttribute**<?>... attrs)
static **Path**	**createTempDirectory**(**Path** dir, **String** pref **FileAttribute**<?>... attrs)
static **Path**	**createTempDirectory**(**String** prefix, **FileAttribute**<?>... attrs)
static **Path**	**createTempFile**(**Path** dir, **String** prefix, **String** suffix, **FileAttribute**<?>... attrs)

static **Path**	**createTempFile**(**String** prefix, **String** suff **FileAttribute**<?>... attrs)
static void	**delete**(**Path** path)
static boolean	**deleteIfExists**(**Path** path)
static boolean	**exists**(**Path** path, **LinkOption**... options)
static **Stream**<**Path**>	**find**(**Path** start, int maxDepth, **BiPredicate**<**Path**,**BasicFileAttributes**> mat **FileVisitOption**... options)
static **Object**	**getAttribute**(**Path** path, **String** attribute, **LinkOption**... options)
static <V extends **FileAttributeView**> V	**getFileAttributeView**(**Path** path, **Class**<V> **LinkOption**... options)
static **FileStore**	**getFileStore**(**Path** path)
static **FileTime**	**getLastModifiedTime**(**Path** path, **LinkOption**... options)
static **UserPrincipal**	**getOwner**(**Path** path, **LinkOption**... options

static **Set**<**PosixFilePermission**>	**getPosixFilePermissions**(**Path** path, **LinkOption**... options)
static boolean	**isDirectory**(**Path** path, **LinkOption**... opti
static boolean	**isExecutable**(**Path** path)
static boolean	**isHidden**(**Path** path)
static boolean	**isReadable**(**Path** path)
static boolean	**isRegularFile**(**Path** path, **LinkOption**... options)
static boolean	**isSameFile**(**Path** path, **Path** path2)
static boolean	**isSymbolicLink**(**Path** path)
static boolean	**isWritable**(**Path** path)
static **Stream**<**String**>	**lines**(**Path** path)
static **Stream**<**String**>	**lines**(**Path** path, **Charset** cs)
static **Stream**<**Path**>	**list**(**Path** dir)

static **Path**	**move**(**Path** source, **Path** target, **CopyOption**... options)
static **BufferedReader**	**newBufferedReader**(**Path** path)
static **BufferedReader**	**newBufferedReader**(**Path** path, **Charset** cs)
static **BufferedWriter**	**newBufferedWriter**(**Path** path, **Charset** cs, **OpenOption**... options)
static **BufferedWriter**	**newBufferedWriter**(**Path** path, **OpenOption**... options)
static **SeekableByteChannel**	**newByteChannel**(**Path** path, **OpenOption**... options)
static **SeekableByteChannel**	**newByteChannel**(**Path** path, **Set**<? extends **OpenOption**> options, **FileAttribute**<?>... attrs)

static **DirectoryStream**<**Path**>	**newDirectoryStream**(**Path** dir)
static **DirectoryStream**<**Path**>	**newDirectoryStream**(**Path** dir, **DirectoryStream.Filter**<? super **Path**> filt
static **DirectoryStream**<**Path**>	**newDirectoryStream**(**Path** dir, **String** glob)
static **InputStream**	**newInputStream**(**Path** path, **OpenOption**... options)
static **OutputStream**	**newOutputStream**(**Path** path, **OpenOption**... options)
static boolean	**notExists**(**Path** path, **LinkOption**... option
static **String**	**probeContentType**(**Path** path)
static byte[]	**readAllBytes**(**Path** path)

static **List**<**String**>	**readAllLines**(**Path** path)
static **List**<**String**>	**readAllLines**(**Path** path, **Charset** cs)
static <A extends **BasicFileAttributes**> A	**readAttributes**(**Path** path, **Class**<A> type, **LinkOption**... options)
static **Map**<**String**,**Object**>	**readAttributes**(**Path** path, **String** attribut **LinkOption**... options)
static **Path**	**readSymbolicLink**(**Path** link)
static **Path**	**setAttribute**(**Path** path, **String** attribute, **Object** value, **LinkOption**... options)
static **Path**	**setLastModifiedTime**(**Path** path, **FileTime** t
static **Path**	**setOwner**(**Path** path, **UserPrincipal** owner)
static **Path**	**setPosixFilePermissions**(**Path** path, **Set**<**PosixFilePermission**> perms)
static long	**size**(**Path** path)
static **Stream**<**Path**>	**walk**(**Path** start, **FileVisitOption**... optio
static **Stream**<**Path**>	**walk**(**Path** start, int maxDepth, **FileVisitOption**... options)

static **Path**	**walkFileTree**(**Path** start, **FileVisitor**<? su **Path**> visitor)
static **Path**	**walkFileTree**(**Path** start, **Set**<**FileVisitOption**> options, int maxDept **FileVisitor**<? super **Path**> visitor)
static **Path**	**write**(**Path** path, byte[] bytes, **OpenOption**... options)
static **Path**	**write**(**Path** path, **Iterable**<? extends **CharSequence**> lines, **Charset** cs, **OpenOption**... options)
static **Path**	**write**(**Path** path, **Iterable**<? extends **CharSequence**> lines, **OpenOption**... option

Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Method Detail

newInputStream

```
public static InputStream newInputStream(Path path,
                                         OpenOption... options)
                                  throws IOException
```

Opens a file, returning an input stream to read from the file. The stream will not be buffered, and is not required to support the `mark` or `reset` methods. The stream will be safe for access by multiple concurrent threads. Reading commences at the beginning of the file. Whether the returned stream is *asynchronously closeable* and/or *interruptible* is highly file system provider specific and therefore not specified.

The `options` parameter determines how the file is opened. If no options are present then it is equivalent to opening the file with the `READ` option. In addition to the `READ` option, an implementation may also support additional implementation specific options.

Parameters:

path - the path to the file to open

options - options specifying how the file is opened

Returns:

a new input stream

Throws:

IllegalArgumentException - if an invalid combination of options is specified

UnsupportedOperationException - if an unsupported option is specified

IOException - if an I/O error occurs

SecurityException - In the case of the default provider, and a security manager is installed, the checkRead method is invoked to check read access to the file.

newOutputStream

```
public static OutputStream newOutputStream(Path path,
                                           OpenOption... options)
                                    throws IOException
```

Opens or creates a file, returning an output stream that may be used to write bytes to the file. The resulting stream will not be buffered. The stream will be safe for access by multiple concurrent threads. Whether the returned stream is *asynchronously closeable* and/or *interruptible* is highly file system provider specific and therefore not specified.

This method opens or creates a file in exactly the manner specified by the newByteChannel method with the exception that the READ option may not be present in the array of options. If no options are present then this method works as if the CREATE, TRUNCATE_EXISTING, and WRITE options are present. In other words, it opens the file for writing, creating the file if it doesn't exist, or initially truncating an existing regular-file to a size of 0 if it exists.

Usage Examples:

```
Path path = ...

// truncate and overwrite an existing file, or create the file if
// it doesn't initially exist
OutputStream out = Files.newOutputStream(path);

// append to an existing file, fail if the file does not exist
out = Files.newOutputStream(path, APPEND);

// append to an existing file, create file if it doesn't initially exist
```

```
    out = Files.newOutputStream(path, CREATE, APPEND);

    // always create new file, failing if it already exists
    out = Files.newOutputStream(path, CREATE_NEW);
```

Parameters:

path - the path to the file to open or create

options - options specifying how the file is opened

Returns:

a new output stream

Throws:

IllegalArgumentException - if options contains an invalid combination of options

UnsupportedOperationException - if an unsupported option is specified

IOException - if an I/O error occurs

SecurityException - In the case of the default provider, and a security manager is installed, the checkWrite method is invoked to check write access to the file. The checkDelete method is invoked to check delete access if the file is opened with the DELETE_ON_CLOSE option.

newByteChannel

```
public static SeekableByteChannel newByteChannel(Path path,
                                                 Set<? extends OpenOption> optior
                                                 FileAttribute<?>... attrs)
                                          throws IOException
```

Opens or creates a file, returning a seekable byte channel to access the file.

The options parameter determines how the file is opened. The READ and WRITE options determine if the file should be opened for reading and/or writing. If neither option (or the APPEND option) is present then the file is opened for reading. By default reading or writing commence at the beginning of the file.

In the addition to READ and WRITE, the following options may be present:

Option	Description
APPEND	If this option is present then the file is opened for writing and each invocation of the channel's write method first advances the position to the end of the file and then writes the requested data. Whether the

	advancement of the position and the writing of the data are done in a single atomic operation is system-dependent and therefore unspecified. This option may not be used in conjunction with the `READ` or `TRUNCATE_EXISTING` options.
`TRUNCATE_EXISTING`	If this option is present then the existing file is truncated to a size of 0 bytes. This option is ignored when the file is opened only for reading.
`CREATE_NEW`	If this option is present then a new file is created, failing if the file already exists or is a symbolic link. When creating a file the check for the existence of the file and the creation of the file if it does not exist is atomic with respect to other file system operations. This option is ignored when the file is opened only for reading.
`CREATE`	If this option is present then an existing file is opened if it exists, otherwise a new file is created. This option is ignored if the `CREATE_NEW` option is also present or the file is opened only for reading.
`DELETE_ON_CLOSE`	When this option is present then the implementation makes a *best effort* attempt to delete the file when closed by the `close` method. If the `close` method is not invoked then a *best effort* attempt is made to delete the file when the Java virtual machine terminates.
`SPARSE`	When creating a new file this option is a *hint* that the new file will be sparse. This option is ignored when not creating a new file.
`SYNC`	Requires that every update to the file's content or metadata be written synchronously to the underlying storage device. (see Synchronized I/O file integrity).
`DSYNC`	Requires that every update to the file's content be written synchronously to the underlying storage device. (see Synchronized I/O file integrity).

An implementation may also support additional implementation specific options.

The `attrs` parameter is optional `file-attributes` to set atomically when a new file is created.

In the case of the default provider, the returned seekable byte channel is a `FileChannel`.

Usage Examples:

```
Path path = ...

// open file for reading
ReadableByteChannel rbc = Files.newByteChannel(path, EnumSet.of(READ)));
```

```
     // open file for writing to the end of an existing file, creating
     // the file if it doesn't already exist
     WritableByteChannel wbc = Files.newByteChannel(path, EnumSet.of(CREATE,APPEN

     // create file with initial permissions, opening it for both reading and wri
      FileAttribute<Set<PosixFilePermission>> perms = ...
     SeekableByteChannel sbc = Files.newByteChannel(path, EnumSet.of(CREATE_NEW,R
```

Parameters:

path - the path to the file to open or create

options - options specifying how the file is opened

attrs - an optional list of file attributes to set atomically when creating the file

Returns:

a new seekable byte channel

Throws:

IllegalArgumentException - if the set contains an invalid combination of options

UnsupportedOperationException - if an unsupported open option is specified or the array contains attributes that cannot be set atomically when creating the file

FileAlreadyExistsException - if a file of that name already exists and the CREATE_NEW option is specified *(optional specific exception)*

IOException - if an I/O error occurs

SecurityException - In the case of the default provider, and a security manager is installed, the checkRead method is invoked to check read access to the path if the file is opened for reading. The checkWrite method is invoked to check write access to the path if the file is opened for writing. The checkDelete method is invoked to check delete access if the file is opened with the DELETE_ON_CLOSE option.

See Also:

FileChannel.open(Path,Set,FileAttribute[])

newByteChannel

```
public static SeekableByteChannel newByteChannel(Path path,
                                                 OpenOption... options)
                                          throws IOException
```

Opens or creates a file, returning a seekable byte channel to access the file.

This method opens or creates a file in exactly the manner specified by the newByteChannel method.

Parameters:

path - the path to the file to open or create

options - options specifying how the file is opened

Returns:

a new seekable byte channel

Throws:

IllegalArgumentException - if the set contains an invalid combination of options

UnsupportedOperationException - if an unsupported open option is specified

FileAlreadyExistsException - if a file of that name already exists and the CREATE_NEW option is specified *(optional specific exception)*

IOException - if an I/O error occurs

SecurityException - In the case of the default provider, and a security manager is installed, the checkRead method is invoked to check read access to the path if the file is opened for reading. The checkWrite method is invoked to check write access to the path if the file is opened for writing. The checkDelete method is invoked to check delete access if the file is opened with the DELETE_ON_CLOSE option.

See Also:

FileChannel.open(Path,OpenOption[])

newDirectoryStream

```
public static DirectoryStream<Path> newDirectoryStream(Path dir)
                                                throws IOException
```

Opens a directory, returning a DirectoryStream to iterate over all entries in the directory. The elements returned by the directory stream's iterator are of type Path, each one representing an entry in the directory. The Path objects are obtained as if by resolving the name of the directory entry against dir.

When not using the try-with-resources construct, then directory stream's close method should be invoked after iteration is completed so as to free any resources held for the open directory.

When an implementation supports operations on entries in the directory that execute in a race-free manner then the returned directory stream is a SecureDirectoryStream.

Parameters:

dir - the path to the directory

Returns:

a new and open DirectoryStream object

Throws:

NotDirectoryException - if the file could not otherwise be opened because it is not a directory *(optional specific exception)*

IOException - if an I/O error occurs

SecurityException - In the case of the default provider, and a security manager is installed, the checkRead method is invoked to check read access to the directory.

newDirectoryStream

```
public static DirectoryStream<Path> newDirectoryStream(Path dir,
                                                       String glob)
                                                throws IOException
```

Opens a directory, returning a DirectoryStream to iterate over the entries in the directory. The elements returned by the directory stream's iterator are of type Path, each one representing an entry in the directory. The Path objects are obtained as if by resolving the name of the directory entry against dir. The entries returned by the iterator are filtered by matching the String representation of their file names against the given *globbing* pattern.

For example, suppose we want to iterate over the files ending with ".java" in a directory:

```
Path dir = ...
try (DirectoryStream<Path> stream = Files.newDirectoryStream(dir, "*.java"))
    :
}
```

The globbing pattern is specified by the getPathMatcher method.

When not using the try-with-resources construct, then directory stream's close method should be invoked after iteration is completed so as to free any resources held for the open directory.

When an implementation supports operations on entries in the directory that execute in a race-free manner then the returned directory stream is a SecureDirectoryStream.

Parameters:

`dir` - the path to the directory

`glob` - the glob pattern

Returns:

a new and open DirectoryStream object

Throws:

`PatternSyntaxException` - if the pattern is invalid

`NotDirectoryException` - if the file could not otherwise be opened because it is not a directory *(optional specific exception)*

`IOException` - if an I/O error occurs

`SecurityException` - In the case of the default provider, and a security manager is installed, the `checkRead` method is invoked to check read access to the directory.

newDirectoryStream

```
public static DirectoryStream<Path> newDirectoryStream(Path dir,
                                                       DirectoryStream.Filter<? s
                                                throws IOException
```

Opens a directory, returning a `DirectoryStream` to iterate over the entries in the directory. The elements returned by the directory stream's `iterator` are of type `Path`, each one representing an entry in the directory. The `Path` objects are obtained as if by `resolving` the name of the directory entry against `dir`. The entries returned by the iterator are filtered by the given `filter`.

When not using the try-with-resources construct, then directory stream's `close` method should be invoked after iteration is completed so as to free any resources held for the open directory.

Where the filter terminates due to an uncaught error or runtime exception then it is propagated to the `hasNext` or `next` method. Where an `IOException` is thrown, it results in the `hasNext` or `next` method throwing a `DirectoryIteratorException` with the `IOException` as the cause.

When an implementation supports operations on entries in the directory that execute in a race-free manner then the returned directory stream is a `SecureDirectoryStream`.

Usage Example: Suppose we want to iterate over the files in a directory that are larger than 8K.

```
DirectoryStream.Filter<Path> filter = new DirectoryStream.Filter<Path>() {
    public boolean accept(Path file) throws IOException {
        return (Files.size(file) > 8192L);
```

```
        }
    };
    Path dir = ...
    try (DirectoryStream<Path> stream = Files.newDirectoryStream(dir, filter)) {
        :
    }
```

Parameters:

`dir` - the path to the directory

`filter` - the directory stream filter

Returns:

a new and open DirectoryStream object

Throws:

`NotDirectoryException` - if the file could not otherwise be opened because it is not a directory *(optional specific exception)*

`IOException` - if an I/O error occurs

`SecurityException` - In the case of the default provider, and a security manager is installed, the `checkRead` method is invoked to check read access to the directory.

createFile

```
public static Path createFile(Path path,
                              FileAttribute<?>... attrs)
                       throws IOException
```

Creates a new and empty file, failing if the file already exists. The check for the existence of the file and the creation of the new file if it does not exist are a single operation that is atomic with respect to all other filesystem activities that might affect the directory.

The `attrs` parameter is optional `file-attributes` to set atomically when creating the file. Each attribute is identified by its `name`. If more than one attribute of the same name is included in the array then all but the last occurrence is ignored.

Parameters:

`path` - the path to the file to create

`attrs` - an optional list of file attributes to set atomically when creating the file

Returns:

the file

Throws:

UnsupportedOperationException - if the array contains an attribute that cannot be set atomically when creating the file

FileAlreadyExistsException - if a file of that name already exists *(optional specific exception)*

IOException - if an I/O error occurs or the parent directory does not exist

SecurityException - In the case of the default provider, and a security manager is installed, the checkWrite method is invoked to check write access to the new file.

createDirectory

```
public static Path createDirectory(Path dir,
                                   FileAttribute<?>... attrs)
                            throws IOException
```

Creates a new directory. The check for the existence of the file and the creation of the directory if it does not exist are a single operation that is atomic with respect to all other filesystem activities that might affect the directory. The `createDirectories` method should be used where it is required to create all nonexistent parent directories first.

The `attrs` parameter is optional `file-attributes` to set atomically when creating the directory. Each attribute is identified by its `name`. If more than one attribute of the same name is included in the array then all but the last occurrence is ignored.

Parameters:

`dir - the directory to create`

`attrs - an optional list of file attributes to set atomically when creating the directory`

Returns:

`the directory`

Throws:

`UnsupportedOperationException - if the array contains an attribute that cannot be set atomically when creating the directory`

`FileAlreadyExistsException - if a directory could not otherwise be created because a file of that name already exists (optional specific exception)`

`IOException - if an I/O error occurs or the parent directory does not exist`

`SecurityException - In the case of the default provider, and a security manager is installed, the checkWrite method is invoked to check write access to the new directory.`

createDirectories

```
public static Path createDirectories(Path dir,
                                     FileAttribute<?>... attrs)
                              throws IOException
```

Creates a directory by creating all nonexistent parent directories first. Unlike the `createDirectory` method, an exception is not thrown if the directory could not be created because it already exists.

The attrs parameter is optional file-attributes to set atomically when creating the nonexistent directories. Each file attribute is identified by its name. If more than one attribute of the same name is included in the array then all but the last occurrence is ignored.

If this method fails, then it may do so after creating some, but not all, of the parent directories.

Parameters:

dir - the directory to create

attrs - an optional list of file attributes to set atomically when creating the directory

Returns:

the directory

Throws:

UnsupportedOperationException - if the array contains an attribute that cannot be set atomically when creating the directory

FileAlreadyExistsException - if dir exists but is not a directory *(optional specific exception)*

IOException - if an I/O error occurs

SecurityException - in the case of the default provider, and a security manager is installed, the checkWrite method is invoked prior to attempting to create a directory and its checkRead is invoked for each parent directory that is checked. If dir is not an absolute path then its toAbsolutePath may need to be invoked to get its absolute path. This may invoke the security manager's checkPropertyAccess method to check access to the system property user.dir

createTempFile

```
public static Path createTempFile(Path dir,
                                  String prefix,
                                  String suffix,
                                  FileAttribute<?>... attrs)
                           throws IOException
```

Creates a new empty file in the specified directory, using the given prefix and suffix strings to generate its name. The resulting Path is associated with the same FileSystem as the given directory.

The details as to how the name of the file is constructed is implementation dependent and therefore not specified. Where possible the prefix and suffix are used to construct candidate names in the same manner as the File.createTempFile(String,String,File) method.

As with the File.createTempFile methods, this method is only part of a temporary-file facility. Where used as a *work files*, the resulting file may be opened using the DELETE_ON_CLOSE option so that the file is deleted when the appropriate close method is invoked. Alternatively, a shutdown-hook, or the File.deleteOnExit() mechanism may be used to delete the file automatically.

The attrs parameter is optional file-attributes to set atomically when creating the file. Each attribute is identified by its name. If more than one attribute of the same name is included in the array then all but the last occurrence is ignored. When no file attributes are specified, then the resulting file may have more restrictive access permissions to files created by the File.createTempFile(String,String,File) method.

Parameters:

dir - the path to directory in which to create the file

prefix - the prefix string to be used in generating the file's name; may be null

suffix - the suffix string to be used in generating the file's name; may be null, in which case ".tmp" is used

attrs - an optional list of file attributes to set atomically when creating the file

Returns:

the path to the newly created file that did not exist before this method was invoked

Throws:

IllegalArgumentException - if the prefix or suffix parameters cannot be used to generate a candidate file name

UnsupportedOperationException - if the array contains an attribute that cannot be set atomically when creating the directory

IOException - if an I/O error occurs or dir does not exist

SecurityException - In the case of the default provider, and a security manager is installed, the checkWrite method is invoked to check write access to the file.

createTempFile

```
public static Path createTempFile(String prefix,
                                  String suffix,
                                  FileAttribute<?>... attrs)
                           throws IOException
```

Creates an empty file in the default temporary-file directory, using the given prefix and suffix to generate its name. The resulting `Path` is associated with the default `FileSystem`.

This method works in exactly the manner specified by the `createTempFile(Path,String,String,FileAttribute[])` method for the case that the `dir` parameter is the temporary-file directory.

Parameters:

`prefix - the prefix string to be used in generating the file's name; may be null`

`suffix - the suffix string to be used in generating the file's name; may be null, in which case ".tmp" is used`

`attrs - an optional list of file attributes to set atomically when creating the file`

Returns:

`the path to the newly created file that did not exist before this method was invoked`

Throws:

`IllegalArgumentException - if the prefix or suffix parameters cannot be used to generate a candidate file name`

`UnsupportedOperationException - if the array contains an attribute that cannot be set atomically when creating the directory`

`IOException - if an I/O error occurs or the temporary-file directory does not exist`

`SecurityException - In the case of the default provider, and a security manager is installed, the checkWrite method is invoked to check write access to the file.`

createTempDirectory

```
public static Path createTempDirectory(Path dir,
                                       String prefix,
                                       FileAttribute<?>... attrs)
                                throws IOException
```

Creates a new directory in the specified directory, using the given prefix to generate its name. The resulting `Path` is associated with the same `FileSystem` as the given directory.

The details as to how the name of the directory is constructed is implementation dependent and therefore not specified. Where possible the `prefix` is used to construct candidate names.

As with the createTempFile methods, this method is only part of a temporary-file facility. A shutdown-hook, or the File.deleteOnExit() mechanism may be used to delete the directory automatically.

The attrs parameter is optional file-attributes to set atomically when creating the directory. Each attribute is identified by its name. If more than one attribute of the same name is included in the array then all but the last occurrence is ignored.

Parameters:

dir - the path to directory in which to create the directory

prefix - the prefix string to be used in generating the directory's name; may be null

attrs - an optional list of file attributes to set atomically when creating the directory

Returns:

the path to the newly created directory that did not exist before this method was invoked

Throws:

IllegalArgumentException - if the prefix cannot be used to generate a candidate directory name

UnsupportedOperationException - if the array contains an attribute that cannot be set atomically when creating the directory

IOException - if an I/O error occurs or dir does not exist

SecurityException - In the case of the default provider, and a security manager is installed, the checkWrite method is invoked to check write access when creating the directory.

createTempDirectory

```
public static Path createTempDirectory(String prefix,
                                       FileAttribute<?>... attrs)
                                throws IOException
```

Creates a new directory in the default temporary-file directory, using the given prefix to generate its name. The resulting Path is associated with the default FileSystem.

This method works in exactly the manner specified by createTempDirectory(Path,String,FileAttribute[]) method for the case that the dir parameter is the temporary-file directory.

Parameters:

prefix - the prefix string to be used in generating the directory's name; may be null

attrs - an optional list of file attributes to set atomically when creating the directory

Returns:

the path to the newly created directory that did not exist before this method was invoked

Throws:

IllegalArgumentException - if the prefix cannot be used to generate a candidate directory name

UnsupportedOperationException - if the array contains an attribute that cannot be set atomically when creating the directory

IOException - if an I/O error occurs or the temporary-file directory does not exist

SecurityException - In the case of the default provider, and a security manager is installed, the checkWrite method is invoked to check write access when creating the directory.

createSymbolicLink

```
public static Path createSymbolicLink(Path link,
                                       Path target,
                                       FileAttribute<?>... attrs)
                                throws IOException
```

Creates a symbolic link to a target *(optional operation)*.

The target parameter is the target of the link. It may be an absolute or relative path and may not exist. When the target is a relative path then file system operations on the resulting link are relative to the path of the link.

The attrs parameter is optional attributes to set atomically when creating the link. Each attribute is identified by its name. If more than one attribute of the same name is included in the array then all but the last occurrence is ignored.

Where symbolic links are supported, but the underlying FileStore does not support symbolic links, then this may fail with an IOException. Additionally, some operating systems may require that the Java virtual machine be started with implementation specific privileges to create symbolic links, in which case this method may throw IOException.

Parameters:

link - the path of the symbolic link to create

target - the target of the symbolic link

attrs - the array of attributes to set atomically when creating the symbolic link

Returns:

the path to the symbolic link

Throws:

UnsupportedOperationException - if the implementation does not support symbolic links or the array contains an attribute that cannot be set atomically when creating the symbolic link

FileAlreadyExistsException - if a file with the name already exists *(optional specific exception)*

IOException - if an I/O error occurs

SecurityException - In the case of the default provider, and a security manager is installed, it denies LinkPermission("symbolic") or its checkWrite method denies write access to the path of the symbolic link.

createLink

```
public static Path createLink(Path link,
                              Path existing)
                       throws IOException
```

Creates a new link (directory entry) for an existing file *(optional operation)*.

The `link` parameter locates the directory entry to create. The `existing` parameter is the path to an existing file. This method creates a new directory entry for the file so that it can be accessed using `link` as the path. On some file systems this is known as creating a "hard link". Whether the file attributes are maintained for the file or for each directory entry is file system specific and therefore not specified. Typically, a file system requires that all links (directory entries) for a file be on the same file system. Furthermore, on some platforms, the Java virtual machine may require to be started with implementation specific privileges to create hard links or to create links to directories.

Parameters:

link - the link (directory entry) to create

existing - a path to an existing file

Returns:

the path to the link (directory entry)

Throws:

UnsupportedOperationException - if the implementation does not support adding an existing file to a directory

FileAlreadyExistsException - if the entry could not otherwise be created because a file of that name already exists *(optional specific exception)*

IOException - if an I/O error occurs

SecurityException - In the case of the default provider, and a security manager is installed, it denies LinkPermission("hard") or its checkWrite method denies write access to either the link or the existing file.

delete

```
public static void delete(Path path)
                  throws IOException
```

Deletes a file.

An implementation may require to examine the file to determine if the file is a directory. Consequently this method may not be atomic with respect to other file system operations. If the file is a symbolic link then the symbolic link itself, not the final target of the link, is deleted.

If the file is a directory then the directory must be empty. In some implementations a directory has entries for special files or links that are created when the directory is created. In such implementations a directory is considered empty when only the special entries exist. This method can be used with the walkFileTree method to delete a directory and all entries in the directory, or an entire *file-tree* where required.

On some operating systems it may not be possible to remove a file when it is open and in use by this Java virtual machine or other programs.

Parameters:

path - the path to the file to delete

Throws:

NoSuchFileException - if the file does not exist *(optional specific exception)*

DirectoryNotEmptyException - if the file is a directory and could not otherwise be deleted because the directory is not empty *(optional specific exception)*

IOException - if an I/O error occurs

SecurityException - In the case of the default provider, and a security manager is installed, the SecurityManager.checkDelete(String) method is invoked to check delete access to the file

deleteIfExists

```
public static boolean deleteIfExists(Path path)
                              throws IOException
```

Deletes a file if it exists.

As with the `delete(Path)` method, an implementation may need to examine the file to determine if the file is a directory. Consequently this method may not be atomic with respect to other file system operations. If the file is a symbolic link, then the symbolic link itself, not the final target of the link, is deleted.

If the file is a directory then the directory must be empty. In some implementations a directory has entries for special files or links that are created when the directory is created. In such implementations a directory is considered empty when only the special entries exist.

On some operating systems it may not be possible to remove a file when it is open and in use by this Java virtual machine or other programs.

Parameters:

`path` - the path to the file to delete

Returns:

`true if the file was deleted by this method; false if the file could not be deleted because it did not exist`

Throws:

`DirectoryNotEmptyException` - if the file is a directory and could not otherwise be deleted because the directory is not empty *(optional specific exception)*

`IOException` - if an I/O error occurs

`SecurityException` - In the case of the default provider, and a security manager is installed, the `SecurityManager.checkDelete(String)` method is invoked to check delete access to the file.

copy

```
public static Path copy(Path source,
                        Path target,
                        CopyOption... options)
                 throws IOException
```

Copy a file to a target file.

This method copies a file to the target file with the `options` parameter specifying how the copy is performed. By default, the copy fails if the target file already exists or is a symbolic link, except if the source and target are the `same` file, in which case the method completes without copying the file. File attributes are not required to be copied to the target file. If symbolic links are supported, and the file is a symbolic link, then the final target of the link is copied. If the file is a directory then it creates an empty directory in the target location (entries in the directory are not copied). This method can be used with the `walkFileTree` method to copy a directory and all entries in the directory, or an entire *file-tree* where required.

The `options` parameter may include any of the following:

Option	Description
`REPLACE_EXISTING`	If the target file exists, then the target file is replaced if it is not a non-empty directory. If the target file exists and is a symbolic link, then the symbolic link itself, not the target of the link, is replaced.
`COPY_ATTRIBUTES`	Attempts to copy the file attributes associated with this file to the target file. The exact file attributes that are copied is platform and file system dependent and therefore unspecified. Minimally, the `last-modified-time` is copied to the target file if supported by both the source and target file stores. Copying of file timestamps may result in precision loss.
`NOFOLLOW_LINKS`	Symbolic links are not followed. If the file is a symbolic link, then the symbolic link itself, not the target of the link, is copied. It is implementation specific if file attributes can be copied to the new link. In other words, the `COPY_ATTRIBUTES` option may be ignored when copying a symbolic link.

An implementation of this interface may support additional implementation specific options.

Copying a file is not an atomic operation. If an `IOException` is thrown, then it is possible that the target file is incomplete or some of its file attributes have not been copied from the source file. When the `REPLACE_EXISTING` option is specified and the target file exists, then the target file is replaced. The check for the existence of the file and the creation of the new file may not be atomic with respect to other file system activities.

Usage Example: Suppose we want to copy a file into a directory, giving it the same file name as the source file:

```
Path source = ...
Path newdir = ...
Files.copy(source, newdir.resolve(source.getFileName());
```

Parameters:

source - the path to the file to copy

target - the path to the target file (may be associated with a different provider to the source path)

options - options specifying how the copy should be done

Returns:

the path to the target file

Throws:

UnsupportedOperationException - if the array contains a copy option that is not supported

FileAlreadyExistsException - if the target file exists but cannot be replaced because the REPLACE_EXISTING option is not specified *(optional specific exception)*

DirectoryNotEmptyException - the REPLACE_EXISTING option is specified but the file cannot be replaced because it is a non-empty directory *(optional specific exception)*

IOException - if an I/O error occurs

SecurityException - In the case of the default provider, and a security manager is installed, the checkRead method is invoked to check read access to the source file, the checkWrite is invoked to check write access to the target file. If a symbolic link is copied the security manager is invoked to check LinkPermission("symbolic").

move

```
public static Path move(Path source,
                        Path target,
                        CopyOption... options)
                 throws IOException
```

Move or rename a file to a target file.

By default, this method attempts to move the file to the target file, failing if the target file exists except if the source and target are the same file, in which case this method has no effect. If the file is a symbolic link then the symbolic link itself, not the target of the link, is moved. This method may be invoked to move an empty directory. In some implementations a directory has entries for special files or links that are created when the directory is created. In such implementations a directory is considered empty when only the special entries exist. When invoked to move a directory that is not empty then the directory is moved if it does not require moving the entries in the directory. For example, renaming a directory on the same FileStore will usually not require moving the entries in the directory. When moving a

directory requires that its entries be moved then this method fails (by throwing an `IOException`). To move a *file tree* may involve copying rather than moving directories and this can be done using the `copy` method in conjunction with the `Files.walkFileTree` utility method.

The `options` parameter may include any of the following:

Option	Description
`REPLACE_EXISTING`	If the target file exists, then the target file is replaced if it is not a non-empty directory. If the target file exists and is a symbolic link, then the symbolic link itself, not the target of the link, is replaced.
`ATOMIC_MOVE`	The move is performed as an atomic file system operation and all other options are ignored. If the target file exists then it is implementation specific if the existing file is replaced or this method fails by throwing an `IOException`. If the move cannot be performed as an atomic file system operation then `AtomicMoveNotSupportedException` is thrown. This can arise, for example, when the target location is on a different `FileStore` and would require that the file be copied, or target location is associated with a different provider to this object.

An implementation of this interface may support additional implementation specific options.

Moving a file will copy the `last-modified-time` to the target file if supported by both source and target file stores. Copying of file timestamps may result in precision loss. An implementation may also attempt to copy other file attributes but is not required to fail if the file attributes cannot be copied. When the move is performed as a non-atomic operation, and an `IOException` is thrown, then the state of the files is not defined. The original file and the target file may both exist, the target file may be incomplete or some of its file attributes may not been copied from the original file.

Usage Examples: Suppose we want to rename a file to "newname", keeping the file in the same directory:

```
Path source = ...
Files.move(source, source.resolveSibling("newname"));
```

Alternatively, suppose we want to move a file to new directory, keeping the same file name, and replacing any existing file of that name in the directory:

```
Path source = ...
Path newdir = ...
Files.move(source, newdir.resolve(source.getFileName()), REPLACE_EXISTING);
```

Parameters:

source - the path to the file to move

target - the path to the target file (may be associated with a different provider to the source path)

options - options specifying how the move should be done

Returns:

the path to the target file

Throws:

UnsupportedOperationException - if the array contains a copy option that is not supported

FileAlreadyExistsException - if the target file exists but cannot be replaced because the REPLACE_EXISTING option is not specified *(optional specific exception)*

DirectoryNotEmptyException - the REPLACE_EXISTING option is specified but the file cannot be replaced because it is a non-empty directory *(optional specific exception)*

AtomicMoveNotSupportedException - if the options array contains the ATOMIC_MOVE option but the file cannot be moved as an atomic file system operation.

IOException - if an I/O error occurs

SecurityException - In the case of the default provider, and a security manager is installed, the checkWrite method is invoked to check write access to both the source and target file.

readSymbolicLink

```
public static Path readSymbolicLink(Path link)
                             throws IOException
```

Reads the target of a symbolic link *(optional operation)*.

If the file system supports symbolic links then this method is used to read the target of the link, failing if the file is not a symbolic link. The target of the link need not exist. The returned Path object will be associated with the same file system as link.

Parameters:

link - the path to the symbolic link

Returns:

a Path object representing the target of the link

Throws:

UnsupportedOperationException - if the implementation does not support symbolic links

NotLinkException - if the target could otherwise not be read because the file is not a symbolic link *(optional specific exception)*

IOException - if an I/O error occurs

SecurityException - In the case of the default provider, and a security manager is installed, it checks that FilePermission has been granted with the "readlink" action to read the link.

getFileStore

```
public static FileStore getFileStore(Path path)
                               throws IOException
```

Returns the FileStore representing the file store where a file is located.

Once a reference to the FileStore is obtained it is implementation specific if operations on the returned FileStore, or FileStoreAttributeView objects obtained from it, continue to depend on the existence of the file. In particular the behavior is not defined for the case that the file is deleted or moved to a different file store.

Parameters:

path - the path to the file

Returns:

the file store where the file is stored

Throws:

IOException - if an I/O error occurs

SecurityException - In the case of the default provider, and a security manager is installed, the checkRead method is invoked to check read access to the file, and in addition it checks RuntimePermission ("getFileStoreAttributes")

isSameFile

```
public static boolean isSameFile(Path path,
                                 Path path2)
                          throws IOException
```

Tests if two paths locate the same file.

If both Path objects are equal then this method returns true without checking if the file exists. If the two Path objects are associated with

different providers then this method returns `false`. Otherwise, this method checks if both `Path` objects locate the same file, and depending on the implementation, may require to open or access both files.

If the file system and files remain static, then this method implements an equivalence relation for non-null `Paths`.

- It is *reflexive*: for `Path f`, `isSameFile(f,f)` should return `true`.
- It is *symmetric*: for two `Paths f` and `g`, `isSameFile(f,g)` will equal `isSameFile(g,f)`.
- It is *transitive*: for three `Paths f`, `g`, and `h`, if `isSameFile(f,g)` returns `true` and `isSameFile(g,h)` returns `true`, then `isSameFile(f,h)` will return return `true`.

Parameters:

`path - one path to the file`

`path2 - the other path`

Returns:

`true if, and only if, the two paths locate the same file`

Throws:

`IOException - if an I/O error occurs`

`SecurityException - In the case of the default provider, and a security manager is installed, the checkRead method is invoked to check read access to both files.`

See Also:

`BasicFileAttributes.fileKey()`

isHidden

```
public static boolean isHidden(Path path)
                        throws IOException
```

Tells whether or not a file is considered *hidden*. The exact definition of hidden is platform or provider dependent. On UNIX for example a file is considered to be hidden if its name begins with a period character ('.'). On Windows a file is considered hidden if it isn't a directory and the DOS `hidden` attribute is set.

Depending on the implementation this method may require to access the file system to determine if the file is considered hidden.

Parameters:

`path - the path to the file to test`

Returns:

`true if the file is considered hidden`

Throws:

`IOException` - if an I/O error occurs

`SecurityException` - In the case of the default provider, and a security manager is installed, the `checkRead` method is invoked to check read access to the file.

probeContentType

```
public static String probeContentType(Path path)
                               throws IOException
```

Probes the content type of a file.

This method uses the installed `FileTypeDetector` implementations to probe the given file to determine its content type. Each file type detector's `probeContentType` is invoked, in turn, to probe the file type. If the file is recognized then the content type is returned. If the file is not recognized by any of the installed file type detectors then a system-default file type detector is invoked to guess the content type.

A given invocation of the Java virtual machine maintains a system-wide list of file type detectors. Installed file type detectors are loaded using the service-provider loading facility defined by the `ServiceLoader` class. Installed file type detectors are loaded using the system class loader. If the system class loader cannot be found then the extension class loader is used; If the extension class loader cannot be found then the bootstrap class loader is used. File type detectors are typically installed by placing them in a JAR file on the application class path or in the extension directory, the JAR file contains a provider-configuration file named `java.nio.file.spi.FileTypeDetector` in the resource directory `META-INF/services`, and the file lists one or more fully-qualified names of concrete subclass of `FileTypeDetector` that have a zero argument constructor. If the process of locating or instantiating the installed file type detectors fails then an unspecified error is thrown. The ordering that installed providers are located is implementation specific.

The return value of this method is the string form of the value of a Multipurpose Internet Mail Extension (MIME) content type as defined by *RFC 2045: Multipurpose Internet Mail Extensions (MIME) Part One: Format of Internet Message Bodies*. The string is guaranteed to be parsable according to the grammar in the RFC.

Parameters:

`path` - the path to the file to probe

Returns:

The content type of the file, or null if the content type cannot be determined

Throws:

`IOException` - if an I/O error occurs

`SecurityException` - If a security manager is installed and it denies an unspecified permission required by a file type detector implementation.

getFileAttributeView

```
public static <V extends FileAttributeView> V getFileAttributeView(Path path,
                                                                   Class<V> type,
                                                                   LinkOption...
```

Returns a file attribute view of a given type.

A file attribute view provides a read-only or updatable view of a set of file attributes. This method is intended to be used where the file attribute view defines type-safe methods to read or update the file attributes. The `type` parameter is the type of the attribute view required and the method returns an instance of that type if supported. The `BasicFileAttributeView` type supports access to the basic attributes of a file. Invoking this method to select a file attribute view of that type will always return an instance of that class.

The `options` array may be used to indicate how symbolic links are handled by the resulting file attribute view for the case that the file is a symbolic link. By default, symbolic links are followed. If the option `NOFOLLOW_LINKS` is present then symbolic links are not followed. This option is ignored by implementations that do not support symbolic links.

Usage Example: Suppose we want read or set a file's ACL, if supported:

```
Path path = ...
AclFileAttributeView view = Files.getFileAttributeView(path, AclFileAttribut
if (view != null) {
    List<AclEntry> acl = view.getAcl();
    :
}
```

Type Parameters:

`V` - The FileAttributeView type

Parameters:

`path` - the path to the file

`type` - the Class object corresponding to the file attribute view

`options` - options indicating how symbolic links are handled

Returns:

a file attribute view of the specified type, or null if the attribute view type is not available

readAttributes

```
public static <A extends BasicFileAttributes> A readAttributes(Path path,
                                                               Class<A> type,
                                                               LinkOption... opti
                                                        throws IOException
```

Reads a file's attributes as a bulk operation.

The `type` parameter is the type of the attributes required and this method returns an instance of that type if supported. All implementations support a basic set of file attributes and so invoking this method with a `type` parameter of `BasicFileAttributes.class` will not throw `UnsupportedOperationException`.

The `options` array may be used to indicate how symbolic links are handled for the case that the file is a symbolic link. By default, symbolic links are followed and the file attribute of the final target of the link is read. If the option `NOFOLLOW_LINKS` is present then symbolic links are not followed.

It is implementation specific if all file attributes are read as an atomic operation with respect to other file system operations.

Usage Example: Suppose we want to read a file's attributes in bulk:

```
Path path = ...
BasicFileAttributes attrs = Files.readAttributes(path, BasicFileAttributes.cl
```

Alternatively, suppose we want to read file's POSIX attributes without following symbolic links:

```
PosixFileAttributes attrs = Files.readAttributes(path, PosixFileAttributes.cl
```

Type Parameters:

`A - The BasicFileAttributes type`

Parameters:

`path - the path to the file`

`type - the Class of the file attributes required to read`

`options - options indicating how symbolic links are handled`

Returns:

`the file attributes`

Throws:

UnsupportedOperationException - if an attributes of the given type are not supported

IOException - if an I/O error occurs

SecurityException - In the case of the default provider, a security manager is installed, its checkRead method is invoked to check read access to the file. If this method is invoked to read security sensitive attributes then the security manager may be invoke to check for additional permissions.

setAttribute

```
public static Path setAttribute(Path path,
                                String attribute,
                                Object value,
                                LinkOption... options)
                         throws IOException
```

Sets the value of a file attribute.

The attribute parameter identifies the attribute to be set and takes the form:

[*view-name*:]*attribute-name*

where square brackets [...] delineate an optional component and the character ':' stands for itself.

view-name is the name of a FileAttributeView that identifies a set of file attributes. If not specified then it defaults to "basic", the name of the file attribute view that identifies the basic set of file attributes common to many file systems. *attribute-name* is the name of the attribute within the set.

The options array may be used to indicate how symbolic links are handled for the case that the file is a symbolic link. By default, symbolic links are followed and the file attribute of the final target of the link is set. If the option NOFOLLOW_LINKS is present then symbolic links are not followed.

Usage Example: Suppose we want to set the DOS "hidden" attribute:

```
Path path = ...
Files.setAttribute(path, "dos:hidden", true);
```

Parameters:

path - the path to the file

attribute - the attribute to set

value - the attribute value

options - options indicating how symbolic links are handled

Returns:

the path parameter

Throws:

UnsupportedOperationException - if the attribute view is not available

IllegalArgumentException - if the attribute name is not specified, or is not recognized, or the attribute value is of the correct type but has an inappropriate value

ClassCastException - if the attribute value is not of the expected type or is a collection containing elements that are not of the expected type

IOException - if an I/O error occurs

SecurityException - In the case of the default provider, and a security manager is installed, its checkWrite method denies write access to the file. If this method is invoked to set security sensitive attributes then the security manager may be invoked to check for additional permissions.

getAttribute

```
public static Object getAttribute(Path path,
                                  String attribute,
                                  LinkOption... options)
                           throws IOException
```

Reads the value of a file attribute.

The attribute parameter identifies the attribute to be read and takes the form:

[*view-name*:]*attribute-name*

where square brackets [...] delineate an optional component and the character ':' stands for itself.

view-name is the name of a FileAttributeView that identifies a set of file attributes. If not specified then it defaults to "basic", the name of the file attribute view that identifies the basic set of file attributes common to many file systems. *attribute-name* is the name of the attribute.

The options array may be used to indicate how symbolic links are handled for the case that the file is a symbolic link. By default, symbolic links are followed and the file attribute of the final target of the link is read. If the option NOFOLLOW_LINKS is present then symbolic links are not followed.

Usage Example: Suppose we require the user ID of the file owner on a system that supports a "unix" view:

```
    Path path = ...
    int uid = (Integer)Files.getAttribute(path, "unix:uid");
```

Parameters:

path - the path to the file

attribute - the attribute to read

options - options indicating how symbolic links are handled

Returns:

the attribute value

Throws:

UnsupportedOperationException - if the attribute view is not available

IllegalArgumentException - if the attribute name is not specified or is not recognized

IOException - if an I/O error occurs

SecurityException - In the case of the default provider, and a security manager is installed, its checkRead method denies read access to the file. If this method is invoked to read security sensitive attributes then the security manager may be invoked to check for additional permissions.

readAttributes

```
public static Map<String,Object> readAttributes(Path path,
                                                 String attributes,
                                                 LinkOption... options)
                                          throws IOException
```

Reads a set of file attributes as a bulk operation.

The attributes parameter identifies the attributes to be read and takes the form:

[*view-name*:]*attribute-list*

where square brackets [...] delineate an optional component and the character ':' stands for itself.

view-name is the name of a FileAttributeView that identifies a set of file attributes. If not specified then it defaults to "basic", the name of the file attribute view that identifies the basic set of file attributes common to many file systems.

The *attribute-list* component is a comma separated list of zero or more names of attributes to read. If the list contains the value "*" then all

attributes are read. Attributes that are not supported are ignored and will not be present in the returned map. It is implementation specific if all attributes are read as an atomic operation with respect to other file system operations.

The following examples demonstrate possible values for the `attributes` parameter:

`"*"`	Read all `basic-file-attributes`.
`"size,lastModifiedTime,lastAccessTime"`	Reads the file size, last modified time, and last access time attributes.
`"posix:*"`	Read all `POSIX-file-attributes`.
`"posix:permissions,owner,size"`	Reads the POSX file permissions, owner, and file size.

The `options` array may be used to indicate how symbolic links are handled for the case that the file is a symbolic link. By default, symbolic links are followed and the file attribute of the final target of the link is read. If the option `NOFOLLOW_LINKS` is present then symbolic links are not followed.

Parameters:

`path - the path to the file`

`attributes - the attributes to read`

`options - options indicating how symbolic links are handled`

Returns:

`a map of the attributes returned; The map's keys are the attribute names, its values are the attribute values`

Throws:

`UnsupportedOperationException - if the attribute view is not available`

`IllegalArgumentException - if no attributes are specified or an unrecognized attributes is specified`

`IOException - if an I/O error occurs`

`SecurityException - In the case of the default provider, and a security manager is installed, its checkRead method denies read access to the file. If this method is invoked to read security sensitive attributes then the security manager may be invoke to check for additional permissions.`

getPosixFilePermissions

```
public static Set<PosixFilePermission> getPosixFilePermissions(Path path,
                                                               LinkOption... opti
                                                        throws IOException
```

Returns a file's POSIX file permissions.

The `path` parameter is associated with a `FileSystem` that supports the `PosixFileAttributeView`. This attribute view provides access to file attributes commonly associated with files on file systems used by operating systems that implement the Portable Operating System Interface (POSIX) family of standards.

The `options` array may be used to indicate how symbolic links are handled for the case that the file is a symbolic link. By default, symbolic links are followed and the file attribute of the final target of the link is read. If the option `NOFOLLOW_LINKS` is present then symbolic links are not followed.

Parameters:

`path` - the path to the file

`options` - options indicating how symbolic links are handled

Returns:

the file permissions

Throws:

`UnsupportedOperationException` - if the associated file system does not support the PosixFileAttributeView

`IOException` - if an I/O error occurs

`SecurityException` - In the case of the default provider, a security manager is installed, and it denies `RuntimePermission("accessUserInformation")` or its `checkRead` method denies read access to the file.

setPosixFilePermissions

```
public static Path setPosixFilePermissions(Path path,
                                           Set<PosixFilePermission> perms)
                                    throws IOException
```

Sets a file's POSIX permissions.

The `path` parameter is associated with a `FileSystem` that supports the `PosixFileAttributeView`. This attribute view provides access to file attributes commonly associated with files on file systems used by operating

systems that implement the Portable Operating System Interface (POSIX) family of standards.

Parameters:

`path` - The path to the file

`perms` - The new set of permissions

Returns:

The path

Throws:

`UnsupportedOperationException` - if the associated file system does not support the `PosixFileAttributeView`

`ClassCastException` - if the sets contains elements that are not of type `PosixFilePermission`

`IOException` - if an I/O error occurs

`SecurityException` - In the case of the default provider, and a security manager is installed, it denies `RuntimePermission("accessUserInformation")` or its `checkWrite` method denies write access to the file.

getOwner

```
public static UserPrincipal getOwner(Path path,
                                      LinkOption... options)
                               throws IOException
```

Returns the owner of a file.

The `path` parameter is associated with a file system that supports `FileOwnerAttributeView`. This file attribute view provides access to a file attribute that is the owner of the file.

Parameters:

`path` - The path to the file

`options` - options indicating how symbolic links are handled

Returns:

A user principal representing the owner of the file

Throws:

`UnsupportedOperationException` - if the associated file system does not support the `FileOwnerAttributeView`

`IOException` - if an I/O error occurs

`SecurityException` - In the case of the default provider, and a security manager is installed, it denies

RuntimePermission("accessUserInformation") or its checkRead method denies read access to the file.

setOwner

```
public static Path setOwner(Path path,
                            UserPrincipal owner)
                     throws IOException
```

Updates the file owner.

The path parameter is associated with a file system that supports FileOwnerAttributeView. This file attribute view provides access to a file attribute that is the owner of the file.

Usage Example: Suppose we want to make "joe" the owner of a file:

```
Path path = ...
UserPrincipalLookupService lookupService =
    provider(path).getUserPrincipalLookupService();
UserPrincipal joe = lookupService.lookupPrincipalByName("joe");
Files.setOwner(path, joe);
```

Parameters:

path - The path to the file

owner - The new file owner

Returns:

The path

Throws:

UnsupportedOperationException - if the associated file system does not support the FileOwnerAttributeView

IOException - if an I/O error occurs

SecurityException - In the case of the default provider, and a security manager is installed, it denies RuntimePermission("accessUserInformation") or its checkWrite method denies write access to the file.

See Also:

FileSystem.getUserPrincipalLookupService(), UserPrincipalLookupService

isSymbolicLink

```
public static boolean isSymbolicLink(Path path)
```

Tests whether a file is a symbolic link.

Where it is required to distinguish an I/O exception from the case that the file is not a symbolic link then the file attributes can be read with the `readAttributes` method and the file type tested with the `BasicFileAttributes.isSymbolicLink()` method.

Parameters:

`path - The path to the file`

Returns:

`true if the file is a symbolic link; false if the file does not exist, is not a symbolic link, or it cannot be determined if the file is a symbolic link or not.`

Throws:

`SecurityException - In the case of the default provider, and a security manager is installed, its checkRead method denies read access to the file.`

isDirectory

```
public static boolean isDirectory(Path path,
                                  LinkOption... options)
```

Tests whether a file is a directory.

The `options` array may be used to indicate how symbolic links are handled for the case that the file is a symbolic link. By default, symbolic links are followed and the file attribute of the final target of the link is read. If the option `NOFOLLOW_LINKS` is present then symbolic links are not followed.

Where it is required to distinguish an I/O exception from the case that the file is not a directory then the file attributes can be read with the `readAttributes` method and the file type tested with the `BasicFileAttributes.isDirectory()` method.

Parameters:

`path - the path to the file to test`

`options - options indicating how symbolic links are handled`

Returns:

`true if the file is a directory; false if the file does not exist, is not a directory, or it cannot be determined if the file is a directory or not.`

Throws:

SecurityException - In the case of the default provider, and a security manager is installed, its checkRead method denies read access to the file.

isRegularFile

```
public static boolean isRegularFile(Path path,
                                    LinkOption... options)
```

Tests whether a file is a regular file with opaque content.

The options array may be used to indicate how symbolic links are handled for the case that the file is a symbolic link. By default, symbolic links are followed and the file attribute of the final target of the link is read. If the option NOFOLLOW_LINKS is present then symbolic links are not followed.

Where it is required to distinguish an I/O exception from the case that the file is not a regular file then the file attributes can be read with the readAttributes method and the file type tested with the BasicFileAttributes.isRegularFile() method.

Parameters:

path - the path to the file

options - options indicating how symbolic links are handled

Returns:

true if the file is a regular file; false if the file does not exist, is not a regular file, or it cannot be determined if the file is a regular file or not.

Throws:

SecurityException - In the case of the default provider, and a security manager is installed, its checkRead method denies read access to the file.

getLastModifiedTime

```
public static FileTime getLastModifiedTime(Path path,
                                           LinkOption... options)
                                    throws IOException
```

Returns a file's last modified time.

The options array may be used to indicate how symbolic links are handled for the case that the file is a symbolic link. By default, symbolic links are followed and the file attribute of the final target of the link is read. If the option NOFOLLOW_LINKS is present then symbolic links are not followed.

Parameters:

path - the path to the file

options - options indicating how symbolic links are handled

Returns:

a FileTime representing the time the file was last modified, or an implementation specific default when a time stamp to indicate the time of last modification is not supported by the file system

Throws:

IOException - if an I/O error occurs

SecurityException - In the case of the default provider, and a security manager is installed, its checkRead method denies read access to the file.

See Also:

BasicFileAttributes.lastModifiedTime()

setLastModifiedTime

```
public static Path setLastModifiedTime(Path path,
                                       FileTime time)
                                throws IOException
```

Updates a file's last modified time attribute. The file time is converted to the epoch and precision supported by the file system. Converting from finer to coarser granularities result in precision loss. The behavior of this method when attempting to set the last modified time when it is not supported by the file system or is outside the range supported by the underlying file store is not defined. It may or not fail by throwing an IOException.

Usage Example: Suppose we want to set the last modified time to the current time:

```
Path path = ...
FileTime now = FileTime.fromMillis(System.currentTimeMillis());
Files.setLastModifiedTime(path, now);
```

Parameters:

path - the path to the file

time - the new last modified time

Returns:

the path

Throws:

IOException - if an I/O error occurs

SecurityException - In the case of the default provider, the security manager's checkWrite method is invoked to check write access to file

See Also:

BasicFileAttributeView.setTimes(java.nio.file.attribute.FileTime, java.nio.file.attribute.FileTime, java.nio.file.attribute.FileTime)

size

```
public static long size(Path path)
                 throws IOException
```

Returns the size of a file (in bytes). The size may differ from the actual size on the file system due to compression, support for sparse files, or other reasons. The size of files that are not regular files is implementation specific and therefore unspecified.

Parameters:

path - the path to the file

Returns:

the file size, in bytes

Throws:

IOException - if an I/O error occurs

SecurityException - In the case of the default provider, and a security manager is installed, its checkRead method denies read access to the file.

See Also:

BasicFileAttributes.size()

exists

```
public static boolean exists(Path path,
                             LinkOption... options)
```

Tests whether a file exists.

The options parameter may be used to indicate how symbolic links are handled for the case that the file is a symbolic link. By default, symbolic links are followed. If the option NOFOLLOW_LINKS is present then symbolic links are not followed.

Note that the result of this method is immediately outdated. If this method indicates the file exists then there is no guarantee that a subsequence access

will succeed. Care should be taken when using this method in security sensitive applications.

Parameters:

`path - the path to the file to test`

`options - options indicating how symbolic links are handled .`

Returns:

`true if the file exists; false if the file does not exist or its existence cannot be determined.`

Throws:

`SecurityException - In the case of the default provider, the SecurityManager.checkRead(String) is invoked to check read access to the file.`

See Also:

`notExists(java.nio.file.Path, java.nio.file.LinkOption...)`

notExists

```
public static boolean notExists(Path path,
                                LinkOption... options)
```

Tests whether the file located by this path does not exist. This method is intended for cases where it is required to take action when it can be confirmed that a file does not exist.

The `options` parameter may be used to indicate how symbolic links are handled for the case that the file is a symbolic link. By default, symbolic links are followed. If the option `NOFOLLOW_LINKS` is present then symbolic links are not followed.

Note that this method is not the complement of the `exists` method. Where it is not possible to determine if a file exists or not then both methods return `false`. As with the `exists` method, the result of this method is immediately outdated. If this method indicates the file does exist then there is no guarantee that a subsequence attempt to create the file will succeed. Care should be taken when using this method in security sensitive applications.

Parameters:

`path - the path to the file to test`

`options - options indicating how symbolic links are handled`

Returns:

`true if the file does not exist; false if the file exists or its existence cannot be determined`

Throws:

SecurityException - In the case of the default provider, the SecurityManager.checkRead(String) is invoked to check read access to the file.

isReadable

```
public static boolean isReadable(Path path)
```

Tests whether a file is readable. This method checks that a file exists and that this Java virtual machine has appropriate privileges that would allow it open the file for reading. Depending on the implementation, this method may require to read file permissions, access control lists, or other file attributes in order to check the effective access to the file. Consequently, this method may not be atomic with respect to other file system operations.

Note that the result of this method is immediately outdated, there is no guarantee that a subsequent attempt to open the file for reading will succeed (or even that it will access the same file). Care should be taken when using this method in security sensitive applications.

Parameters:

path - the path to the file to check

Returns:

true if the file exists and is readable; false if the file does not exist, read access would be denied because the Java virtual machine has insufficient privileges, or access cannot be determined

Throws:

SecurityException - In the case of the default provider, and a security manager is installed, the checkRead is invoked to check read access to the file.

isWritable

```
public static boolean isWritable(Path path)
```

Tests whether a file is writable. This method checks that a file exists and that this Java virtual machine has appropriate privileges that would allow it open the file for writing. Depending on the implementation, this method may require to read file permissions, access control lists, or other file attributes in order to check the effective access to the file. Consequently, this method may not be atomic with respect to other file system operations.

Note that result of this method is immediately outdated, there is no guarantee that a subsequent attempt to open the file for writing will succeed

(or even that it will access the same file). Care should be taken when using this method in security sensitive applications.

Parameters:

`path - the path to the file to check`

Returns:

`true if the file exists and is writable; false if the file does not exist, write access would be denied because the Java virtual machine has insufficient privileges, or access cannot be determined`

Throws:

`SecurityException - In the case of the default provider, and a security manager is installed, the checkWrite is invoked to check write access to the file.`

isExecutable

```
public static boolean isExecutable(Path path)
```

Tests whether a file is executable. This method checks that a file exists and that this Java virtual machine has appropriate privileges to `execute` the file. The semantics may differ when checking access to a directory. For example, on UNIX systems, checking for execute access checks that the Java virtual machine has permission to search the directory in order to access file or subdirectories.

Depending on the implementation, this method may require to read file permissions, access control lists, or other file attributes in order to check the effective access to the file. Consequently, this method may not be atomic with respect to other file system operations.

Note that the result of this method is immediately outdated, there is no guarantee that a subsequent attempt to execute the file will succeed (or even that it will access the same file). Care should be taken when using this method in security sensitive applications.

Parameters:

`path - the path to the file to check`

Returns:

`true if the file exists and is executable; false if the file does not exist, execute access would be denied because the Java virtual machine has insufficient privileges, or access cannot be determined`

Throws:

SecurityException - In the case of the default provider, and a security manager is installed, the checkExec is invoked to check execute access to the file.

walkFileTree

```
public static Path walkFileTree(Path start,
                                Set<FileVisitOption> options,
                                int maxDepth,
                                FileVisitor<? super Path> visitor)
                         throws IOException
```

Walks a file tree.

This method walks a file tree rooted at a given starting file. The file tree traversal is *depth-first* with the given FileVisitor invoked for each file encountered. File tree traversal completes when all accessible files in the tree have been visited, or a visit method returns a result of TERMINATE. Where a visit method terminates due an IOException, an uncaught error, or runtime exception, then the traversal is terminated and the error or exception is propagated to the caller of this method.

For each file encountered this method attempts to read its BasicFileAttributes. If the file is not a directory then the visitFile method is invoked with the file attributes. If the file attributes cannot be read, due to an I/O exception, then the visitFileFailed method is invoked with the I/O exception.

Where the file is a directory, and the directory could not be opened, then the visitFileFailed method is invoked with the I/O exception, after which, the file tree walk continues, by default, at the next *sibling* of the directory.

Where the directory is opened successfully, then the entries in the directory, and their *descendants* are visited. When all entries have been visited, or an I/O error occurs during iteration of the directory, then the directory is closed and the visitor's postVisitDirectory method is invoked. The file tree walk then continues, by default, at the next *sibling* of the directory.

By default, symbolic links are not automatically followed by this method. If the options parameter contains the FOLLOW_LINKS option then symbolic links are followed. When following links, and the attributes of the target cannot be read, then this method attempts to get the BasicFileAttributes of the link. If they can be read then the visitFile method is invoked with the attributes of the link (otherwise the visitFileFailed method is invoked as specified above).

If the options parameter contains the FOLLOW_LINKS option then this method keeps track of directories visited so that cycles can be detected. A cycle arises when there is an entry in a directory that is an ancestor of the directory. Cycle detection is done by recording the file-key of directories,

or if file keys are not available, by invoking the `isSameFile` method to test if a directory is the same file as an ancestor. When a cycle is detected it is treated as an I/O error, and the `visitFileFailed` method is invoked with an instance of `FileSystemLoopException`.

The `maxDepth` parameter is the maximum number of levels of directories to visit. A value of `0` means that only the starting file is visited, unless denied by the security manager. A value of `MAX_VALUE` may be used to indicate that all levels should be visited. The `visitFile` method is invoked for all files, including directories, encountered at `maxDepth`, unless the basic file attributes cannot be read, in which case the `visitFileFailed` method is invoked.

If a visitor returns a result of `null` then `NullPointerException` is thrown.

When a security manager is installed and it denies access to a file (or directory), then it is ignored and the visitor is not invoked for that file (or directory).

Parameters:

`start - the starting file`

`options - options to configure the traversal`

`maxDepth - the maximum number of directory levels to visit`

`visitor - the file visitor to invoke for each file`

Returns:

`the starting file`

Throws:

`IllegalArgumentException - if the maxDepth parameter is negative`

`SecurityException - If the security manager denies access to the starting file. In the case of the default provider, the checkRead method is invoked to check read access to the directory.`

`IOException - if an I/O error is thrown by a visitor method`

walkFileTree

```
public static Path walkFileTree(Path start,
                                FileVisitor<? super Path> visitor)
                         throws IOException
```

Walks a file tree.

This method works as if invoking it were equivalent to evaluating the expression:

```
walkFileTree(start, EnumSet.noneOf(FileVisitOption.class), Integer.MAX_VALU
```

In other words, it does not follow symbolic links, and visits all levels of the file tree.

Parameters:

`start` - the starting file

`visitor` - the file visitor to invoke for each file

Returns:

the starting file

Throws:

`SecurityException` - If the security manager denies access to the starting file. In the case of the default provider, the `checkRead` method is invoked to check read access to the directory.

`IOException` - if an I/O error is thrown by a visitor method

newBufferedReader

```
public static BufferedReader newBufferedReader(Path path,
                                               Charset cs)
                                        throws IOException
```

Opens a file for reading, returning a `BufferedReader` that may be used to read text from the file in an efficient manner. Bytes from the file are decoded into characters using the specified charset. Reading commences at the beginning of the file.

The `Reader` methods that read from the file throw `IOException` if a malformed or unmappable byte sequence is read.

Parameters:

`path` - the path to the file

`cs` - the charset to use for decoding

Returns:

a new buffered reader, with default buffer size, to read text from the file

Throws:

`IOException` - if an I/O error occurs opening the file

`SecurityException` - In the case of the default provider, and a security manager is installed, the `checkRead` method is invoked to check read access to the file.

See Also:

`readAllLines(java.nio.file.Path, java.nio.charset.Charset)`

newBufferedReader

```
public static BufferedReader newBufferedReader(Path path)
                                        throws IOException
```

Opens a file for reading, returning a `BufferedReader` to read text from the file in an efficient manner. Bytes from the file are decoded into characters using the `UTF-8 charset`.

This method works as if invoking it were equivalent to evaluating the expression:

```
Files.newBufferedReader(path, StandardCharsets.UTF_8)
```

Parameters:

`path - the path to the file`

Returns:

`a new buffered reader, with default buffer size, to read text from the file`

Throws:

`IOException - if an I/O error occurs opening the file`

`SecurityException - In the case of the default provider, and a security manager is installed, the checkRead method is invoked to check read access to the file.`

Since:

`1.8`

newBufferedWriter

```
public static BufferedWriter newBufferedWriter(Path path,
                                               Charset cs,
                                               OpenOption... options)
                                        throws IOException
```

Opens or creates a file for writing, returning a `BufferedWriter` that may be used to write text to the file in an efficient manner. The `options` parameter specifies how the the file is created or opened. If no options are present then this method works as if the `CREATE`, `TRUNCATE_EXISTING`, and `WRITE` options are present. In other words, it opens the file for writing, creating the file if it doesn't exist, or initially truncating an existing `regular-file` to a size of `0` if it exists.

The `Writer` methods to write text throw `IOException` if the text cannot be encoded using the specified charset.

Parameters:

path - the path to the file

cs - the charset to use for encoding

options - options specifying how the file is opened

Returns:

a new buffered writer, with default buffer size, to write text to the file

Throws:

IOException - if an I/O error occurs opening or creating the file

UnsupportedOperationException - if an unsupported option is specified

SecurityException - In the case of the default provider, and a security manager is installed, the checkWrite method is invoked to check write access to the file.

See Also:

write(Path,Iterable,Charset,OpenOption[])

newBufferedWriter

```
public static BufferedWriter newBufferedWriter(Path path,
                                               OpenOption... options)
                                        throws IOException
```

Opens or creates a file for writing, returning a BufferedWriter to write text to the file in an efficient manner. The text is encoded into bytes for writing using the UTF-8 charset.

This method works as if invoking it were equivalent to evaluating the expression:

```
Files.newBufferedWriter(path, StandardCharsets.UTF_8, options)
```

Parameters:

path - the path to the file

options - options specifying how the file is opened

Returns:

a new buffered writer, with default buffer size, to write text to the file

Throws:

IOException - if an I/O error occurs opening or creating the file

UnsupportedOperationException - if an unsupported option is specified

SecurityException - In the case of the default provider, and a security manager is installed, the checkWrite method is invoked to check write access to the file.

Since:

1.8

copy

```
public static long copy(InputStream in,
                        Path target,
                        CopyOption... options)
                 throws IOException
```

Copies all bytes from an input stream to a file. On return, the input stream will be at end of stream.

By default, the copy fails if the target file already exists or is a symbolic link. If the REPLACE_EXISTING option is specified, and the target file already exists, then it is replaced if it is not a non-empty directory. If the target file exists and is a symbolic link, then the symbolic link is replaced. In this release, the REPLACE_EXISTING option is the only option required to be supported by this method. Additional options may be supported in future releases.

If an I/O error occurs reading from the input stream or writing to the file, then it may do so after the target file has been created and after some bytes have been read or written. Consequently the input stream may not be at end of stream and may be in an inconsistent state. It is strongly recommended that the input stream be promptly closed if an I/O error occurs.

This method may block indefinitely reading from the input stream (or writing to the file). The behavior for the case that the input stream is *asynchronously closed* or the thread interrupted during the copy is highly input stream and file system provider specific and therefore not specified.

Usage example: Suppose we want to capture a web page and save it to a file:

```
Path path = ...
URI u = URI.create("http://java.sun.com/");
try (InputStream in = u.toURL().openStream()) {
    Files.copy(in, path);
}
```

Parameters:

in - the input stream to read from

target - the path to the file

options - options specifying how the copy should be done

Returns:

the number of bytes read or written

Throws:

IOException - if an I/O error occurs when reading or writing

FileAlreadyExistsException - if the target file exists but cannot be replaced because the REPLACE_EXISTING option is not specified *(optional specific exception)*

DirectoryNotEmptyException - the REPLACE_EXISTING option is specified but the file cannot be replaced because it is a non-empty directory *(optional specific exception)* *

UnsupportedOperationException - if options contains a copy option that is not supported

SecurityException - In the case of the default provider, and a security manager is installed, the checkWrite method is invoked to check write access to the file. Where the REPLACE_EXISTING option is specified, the security manager's checkDelete method is invoked to check that an existing file can be deleted.

copy

```
public static long copy(Path source,
                        OutputStream out)
                 throws IOException
```

Copies all bytes from a file to an output stream.

If an I/O error occurs reading from the file or writing to the output stream, then it may do so after some bytes have been read or written. Consequently the output stream may be in an inconsistent state. It is strongly recommended that the output stream be promptly closed if an I/O error occurs.

This method may block indefinitely writing to the output stream (or reading from the file). The behavior for the case that the output stream is *asynchronously closed* or the thread interrupted during the copy is highly output stream and file system provider specific and therefore not specified.

Note that if the given output stream is Flushable then its flush method may need to invoked after this method completes so as to flush any buffered output.

Parameters:

source - the path to the file

out - the output stream to write to

Returns:

the number of bytes read or written

Throws:

IOException - if an I/O error occurs when reading or writing

SecurityException - In the case of the default provider, and a security manager is installed, the checkRead method is invoked to check read access to the file.

readAllBytes

```
public static byte[] readAllBytes(Path path)
                           throws IOException
```

Reads all the bytes from a file. The method ensures that the file is closed when all bytes have been read or an I/O error, or other runtime exception, is thrown.

Note that this method is intended for simple cases where it is convenient to read all bytes into a byte array. It is not intended for reading in large files.

Parameters:

path - the path to the file

Returns:

a byte array containing the bytes read from the file

Throws:

IOException - if an I/O error occurs reading from the stream

OutOfMemoryError - if an array of the required size cannot be allocated, for example the file is larger that 2GB

SecurityException - In the case of the default provider, and a security manager is installed, the checkRead method is invoked to check read access to the file.

readAllLines

```
public static List<String> readAllLines(Path path,
                                        Charset cs)
                                 throws IOException
```

Read all lines from a file. This method ensures that the file is closed when all bytes have been read or an I/O error, or other runtime exception, is thrown. Bytes from the file are decoded into characters using the specified charset.

This method recognizes the following as line terminators:

- \u000D followed by \u000A, CARRIAGE RETURN followed by LINE FEED
- \u000A, LINE FEED
- \u000D, CARRIAGE RETURN

Additional Unicode line terminators may be recognized in future releases.

Note that this method is intended for simple cases where it is convenient to read all lines in a single operation. It is not intended for reading in large files.

Parameters:

path - the path to the file

cs - the charset to use for decoding

Returns:

the lines from the file as a List; whether the List is modifiable or not is implementation dependent and therefore not specified

Throws:

IOException - if an I/O error occurs reading from the file or a malformed or unmappable byte sequence is read

SecurityException - In the case of the default provider, and a security manager is installed, the checkRead method is invoked to check read access to the file.

See Also:

newBufferedReader(java.nio.file.Path, java.nio.charset.Charset)

readAllLines

```
public static List<String> readAllLines(Path path)
                                 throws IOException
```

Read all lines from a file. Bytes from the file are decoded into characters using the UTF-8 charset.

This method works as if invoking it were equivalent to evaluating the expression:

```
Files.readAllLines(path, StandardCharsets.UTF_8)
```

Parameters:

path - the path to the file

Returns:

the lines from the file as a List; whether the List is modifiable or not is implementation dependent and therefore not specified

Throws:

IOException - if an I/O error occurs reading from the file or a malformed or unmappable byte sequence is read

SecurityException - In the case of the default provider, and a security manager is installed, the checkRead method is invoked to check read access to the file.

Since:

1.8

write

```
public static Path write(Path path,
                         byte[] bytes,
                         OpenOption... options)
                  throws IOException
```

Writes bytes to a file. The options parameter specifies how the the file is created or opened. If no options are present then this method works as if the CREATE, TRUNCATE_EXISTING, and WRITE options are present. In other words, it opens the file for writing, creating the file if it doesn't exist, or initially truncating an existing regular-file to a size of 0. All bytes in the byte array are written to the file. The method ensures that the file is closed when all bytes have been written (or an I/O error or other runtime exception is thrown). If an I/O error occurs then it may do so after the file has created or truncated, or after some bytes have been written to the file.

Usage example: By default the method creates a new file or overwrites an existing file. Suppose you instead want to append bytes to an existing file:

```
Path path = ...
byte[] bytes = ...
Files.write(path, bytes, StandardOpenOption.APPEND);
```

Parameters:

path - the path to the file

bytes - the byte array with the bytes to write

options - options specifying how the file is opened

Returns:

the path

Throws:

IOException - if an I/O error occurs writing to or creating the file

UnsupportedOperationException - if an unsupported option is specified

SecurityException - In the case of the default provider, and a security manager is installed, the checkWrite method is invoked to check write access to the file.

write

```
public static Path write(Path path,
                         Iterable<? extends CharSequence> lines,
                         Charset cs,
                         OpenOption... options)
                  throws IOException
```

Write lines of text to a file. Each line is a char sequence and is written to the file in sequence with each line terminated by the platform's line separator, as defined by the system property `line.separator`. Characters are encoded into bytes using the specified charset.

The `options` parameter specifies how the the file is created or opened. If no options are present then this method works as if the `CREATE`, `TRUNCATE_EXISTING`, and `WRITE` options are present. In other words, it opens the file for writing, creating the file if it doesn't exist, or initially truncating an existing `regular-file` to a size of `0`. The method ensures that the file is closed when all lines have been written (or an I/O error or other runtime exception is thrown). If an I/O error occurs then it may do so after the file has created or truncated, or after some bytes have been written to the file.

Parameters:

path - the path to the file

lines - an object to iterate over the char sequences

cs - the charset to use for encoding

options - options specifying how the file is opened

Returns:

the path

Throws:

IOException - if an I/O error occurs writing to or creating the file, or the text cannot be encoded using the specified charset

UnsupportedOperationException - if an unsupported option is specified

SecurityException - In the case of the default provider, and a security manager is installed, the checkWrite method is invoked to check write access to the file.

write

```
public static Path write(Path path,
                         Iterable<? extends CharSequence> lines,
                         OpenOption... options)
                  throws IOException
```

Write lines of text to a file. Characters are encoded into bytes using the UTF-8 charset.

This method works as if invoking it were equivalent to evaluating the expression:

```
Files.write(path, lines, StandardCharsets.UTF_8, options);
```

Parameters:

path - the path to the file

lines - an object to iterate over the char sequences

options - options specifying how the file is opened

Returns:

the path

Throws:

IOException - if an I/O error occurs writing to or creating the file, or the text cannot be encoded as UTF-8

UnsupportedOperationException - if an unsupported option is specified

SecurityException - In the case of the default provider, and a security manager is installed, the checkWrite method is invoked to check write access to the file.

Since:

1.8

list

```
public static Stream<Path> list(Path dir)
                         throws IOException
```

Return a lazily populated Stream, the elements of which are the entries in the directory. The listing is not recursive.

The elements of the stream are Path objects that are obtained as if by resolving the name of the directory entry against dir. Some file systems

maintain special links to the directory itself and the directory's parent directory. Entries representing these links are not included.

The stream is *weakly consistent*. It is thread safe but does not freeze the directory while iterating, so it may (or may not) reflect updates to the directory that occur after returning from this method.

The returned stream encapsulates a `DirectoryStream`. If timely disposal of file system resources is required, the `try`-with-resources construct should be used to ensure that the stream's `close` method is invoked after the stream operations are completed.

Operating on a closed stream behaves as if the end of stream has been reached. Due to read-ahead, one or more elements may be returned after the stream has been closed.

If an `IOException` is thrown when accessing the directory after this method has returned, it is wrapped in an `UncheckedIOException` which will be thrown from the method that caused the access to take place.

Parameters:

`dir - The path to the directory`

Returns:

`The Stream describing the content of the directory`

Throws:

`NotDirectoryException - if the file could not otherwise be opened because it is not a directory (optional specific exception)`

`IOException - if an I/O error occurs when opening the directory`

`SecurityException - In the case of the default provider, and a security manager is installed, the checkRead method is invoked to check read access to the directory.`

Since:

`1.8`

See Also:

`newDirectoryStream(Path)`

walk

```
public static Stream<Path> walk(Path start,
                                int maxDepth,
                                FileVisitOption... options)
                         throws IOException
```

Return a `Stream` that is lazily populated with `Path` by walking the file tree rooted at a given starting file. The file tree is traversed *depth-first,* the

elements in the stream are `Path` objects that are obtained as if by `resolving` the relative path against `start`.

The `stream` walks the file tree as elements are consumed. The `Stream` returned is guaranteed to have at least one element, the starting file itself. For each file visited, the stream attempts to read its `BasicFileAttributes`. If the file is a directory and can be opened successfully, entries in the directory, and their *descendants* will follow the directory in the stream as they are encountered. When all entries have been visited, then the directory is closed. The file tree walk then continues at the next *sibling* of the directory.

The stream is *weakly consistent*. It does not freeze the file tree while iterating, so it may (or may not) reflect updates to the file tree that occur after returned from this method.

By default, symbolic links are not automatically followed by this method. If the `options` parameter contains the `FOLLOW_LINKS` option then symbolic links are followed. When following links, and the attributes of the target cannot be read, then this method attempts to get the `BasicFileAttributes` of the link.

If the `options` parameter contains the `FOLLOW_LINKS` option then the stream keeps track of directories visited so that cycles can be detected. A cycle arises when there is an entry in a directory that is an ancestor of the directory. Cycle detection is done by recording the `file-key` of directories, or if file keys are not available, by invoking the `isSameFile` method to test if a directory is the same file as an ancestor. When a cycle is detected it is treated as an I/O error with an instance of `FileSystemLoopException`.

The `maxDepth` parameter is the maximum number of levels of directories to visit. A value of `0` means that only the starting file is visited, unless denied by the security manager. A value of `MAX_VALUE` may be used to indicate that all levels should be visited.

When a security manager is installed and it denies access to a file (or directory), then it is ignored and not included in the stream.

The returned stream encapsulates one or more `DirectoryStream`s. If timely disposal of file system resources is required, the `try`-with-resources construct should be used to ensure that the stream's `close` method is invoked after the stream operations are completed. Operating on a closed stream will result in an `IllegalStateException`.

If an `IOException` is thrown when accessing the directory after this method has returned, it is wrapped in an `UncheckedIOException` which will be thrown from the method that caused the access to take place.

Parameters:

`start - the starting file`

`maxDepth - the maximum number of directory levels to visit`

`options - options to configure the traversal`

Returns:

the Stream of Path

Throws:

IllegalArgumentException - if the maxDepth parameter is negative

SecurityException - If the security manager denies access to the starting file. In the case of the default provider, the checkRead method is invoked to check read access to the directory.

IOException - if an I/O error is thrown when accessing the starting file.

Since:

1.8

walk

```
public static Stream<Path> walk(Path start,
                                FileVisitOption... options)
                         throws IOException
```

Return a Stream that is lazily populated with Path by walking the file tree rooted at a given starting file. The file tree is traversed *depth-first*, the elements in the stream are Path objects that are obtained as if by resolving the relative path against start.

This method works as if invoking it were equivalent to evaluating the expression:

```
walk(start, Integer.MAX_VALUE, options)
```

In other words, it visits all levels of the file tree.

The returned stream encapsulates one or more DirectoryStreams. If timely disposal of file system resources is required, the try-with-resources construct should be used to ensure that the stream's close method is invoked after the stream operations are completed. Operating on a closed stream will result in an IllegalStateException.

Parameters:

start - the starting file

options - options to configure the traversal

Returns:

the Stream of Path

Throws:

SecurityException - If the security manager denies access to the starting file. In the case of the default provider, the checkRead method is invoked to check read access to the directory.

IOException - if an I/O error is thrown when accessing the starting file.

Since:

1.8

See Also:

walk(Path, int, FileVisitOption...)

find

```
public static Stream<Path> find(Path start,
                                int maxDepth,
                                BiPredicate<Path,BasicFileAttributes> matcher,
                                FileVisitOption... options)
                         throws IOException
```

Return a Stream that is lazily populated with Path by searching for files in a file tree rooted at a given starting file.

This method walks the file tree in exactly the manner specified by the walk method. For each file encountered, the given BiPredicate is invoked with its Path and BasicFileAttributes. The Path object is obtained as if by resolving the relative path against start and is only included in the returned Stream if the BiPredicate returns true. Compare to calling filter on the Stream returned by walk method, this method may be more efficient by avoiding redundant retrieval of the BasicFileAttributes.

The returned stream encapsulates one or more DirectoryStreams. If timely disposal of file system resources is required, the try-with-resources construct should be used to ensure that the stream's close method is invoked after the stream operations are completed. Operating on a closed stream will result in an IllegalStateException.

If an IOException is thrown when accessing the directory after returned from this method, it is wrapped in an UncheckedIOException which will be thrown from the method that caused the access to take place.

Parameters:

start - the starting file

maxDepth - the maximum number of directory levels to search

matcher - the function used to decide whether a file should be included in the returned stream

options - options to configure the traversal

Returns:

the Stream of Path

Throws:

IllegalArgumentException - if the maxDepth parameter is negative

SecurityException - If the security manager denies access to the starting file. In the case of the default provider, the checkRead method is invoked to check read access to the directory.

IOException - if an I/O error is thrown when accessing the starting file.

Since:

1.8

See Also:

walk(Path, int, FileVisitOption...)

lines

```
public static Stream<String> lines(Path path,
                                   Charset cs)
                            throws IOException
```

Read all lines from a file as a Stream. Unlike readAllLines, this method does not read all lines into a List, but instead populates lazily as the stream is consumed.

Bytes from the file are decoded into characters using the specified charset and the same line terminators as specified by readAllLines are supported.

After this method returns, then any subsequent I/O exception that occurs while reading from the file or when a malformed or unmappable byte sequence is read, is wrapped in an UncheckedIOException that will be thrown from the Stream method that caused the read to take place. In case an IOException is thrown when closing the file, it is also wrapped as an UncheckedIOException.

The returned stream encapsulates a Reader. If timely disposal of file system resources is required, the try-with-resources construct should be used to ensure that the stream's close method is invoked after the stream operations are completed.

Parameters:

path - the path to the file

cs - the charset to use for decoding

Returns:

the lines from the file as a Stream

Throws:

IOException - if an I/O error occurs opening the file

SecurityException - In the case of the default provider, and a security manager is installed, the checkRead method is invoked to check read access to the file.

Since:

1.8

See Also:

readAllLines(Path, Charset), newBufferedReader(Path, Charset), BufferedReader.lines()

lines

```
public static Stream<String> lines(Path path)
                           throws IOException
```

Read all lines from a file as a Stream. Bytes from the file are decoded into characters using the UTF-8 charset.

This method works as if invoking it were equivalent to evaluating the expression:

```
Files.lines(path, StandardCharsets.UTF_8)
```

Parameters:

path - the path to the file

Returns:

the lines from the file as a Stream

Throws:

IOException - if an I/O error occurs opening the file

SecurityException - In the case of the default provider, and a security manager is installed, the checkRead method is invoked to check read access to the file.

Since:

```
1.8
```


Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.